

Alohomora

Luis F. Recalde[†]

Abstract—The Probability of Boundary (Pb) algorithm offers richer contour representations by integrating responses from multi-scale, multi-orientation filters across brightness, color, and texture channels. This work implements a simplified Pb-lite algorithm that enhances contour detection by applying filter banks to generate gradients from texture, brightness, and color maps, outperforming classical methods. Furthermore, due to the advantages of deep learning over traditional geometric approaches in tasks such as classification and segmentation, this study implements different neural network architectures and analyzes their training accuracy and testing performance, providing valuable information for each architecture.

I. PHASE I: SHAKE MY BOUNDARY

Contour detection is an important and extensively studied technique in computer vision. However, detecting boundaries in the image plane is complex and requires a certain level of object reasoning. Numerous formulations have been developed to quantify the presence of boundaries in an image using local measurements. For instance, the Sobel algorithm introduced in [1] detects edges by applying a local derivative filter across the image. Another well-known formulation, Canny, presented in [2], employs a descriptor capable of modeling image discontinuities.

However, a richer description can be achieved by analyzing the image response to a family of filters at different scales and orientations. This concept is introduced by the probability of boundary (Pb) algorithm [3], which considers multiple channels: brightness, color, and texture of the image. This modern formulation assigns probabilities to potential boundaries, providing a more robust representation of contour information.

In this section, a simplified version of the Pb algorithm (Pb-lite) is presented. This formulation takes into account a bank of filters with different scales and orientations applied across the channels of the image. This approach outperforms classical formulations such as Canny and Sobel. A series of steps should be completed to implement this algorithm, such as *Creation of Filter Banks*, *Creation of Texture, Brightness,*

and Color Maps, and *Texture, Brightness, and Color Gradients*. The output of the Pb-lite algorithm can be computed through the integration of these maps.

A. Creation of Filter Banks

This section outlines the Pb-lite algorithm's filter bank design, incorporating oriented derivative of Gaussian (DoF), the Laung-Malik (LM), and Gabor filters (GB).

1) *Oriented Derivative of Gaussian Filters*: This filter is designed using the first-order time derivative of a 2D Gaussian function expressed as follows:

$$H_x(x, y, \sigma_x, \sigma_y) = -\frac{x}{\sigma_x^2} H(x, y, \sigma_x, \sigma_y) \quad (1)$$

where $H(x, y, \sigma_x, \sigma_y)$ represents the 2D Gaussian function:

$$H(x, y, \sigma_x, \sigma_y) = \frac{1}{2\pi\sigma_x\sigma_y} \exp\left(-\frac{x^2}{2\sigma_x^2} - \frac{y^2}{2\sigma_y^2}\right) \quad (2)$$

Furthermore, to achieve the oriented variant of this filter, the image can be rotated about its center. For this set of filters, this work considers $\sigma_x = \sigma_y$ and $\sigma_x \in \{1, 2\}$. The number of orientations is denoted by N and lies within $0 \leq \theta \leq 2\pi$. Consequently, this filter bank is characterized by the set:

$$\mathcal{F}_x = \{H_x(x, y, \sigma_x, \sigma_y) | \sigma_x \in \{1, 2\}, \theta \in \{0, 2\pi\}\} \quad (3)$$

The filter bank with $N = 16$ orientations is presented in Fig. 1. It should be noted that in order to preserve the intensity of the image and the probability density of the filters, normalization is required.

2) *Leung-Malik Filters*: The Leung-Malik (LM) filters are sets characterized by multiple scales and orientations, amounting to a total of 48. The LM filter bank includes derivatives of Gaussian functions up to the second order, computed across 6 orientations and 3 scales, resulting in 36 filters; 8 Laplacian of Gaussian (LOG) filters; along with 4 Gaussian filters. The second-order derivative of a 2D Gaussian function is written as follows:

$$H_{xx}(x, y, \sigma_x, \sigma_y) = \left(\frac{x^2}{\sigma_x^4} - \frac{1}{\sigma_x^2}\right) H(x, y, \sigma_x, \sigma_y) \quad (4)$$

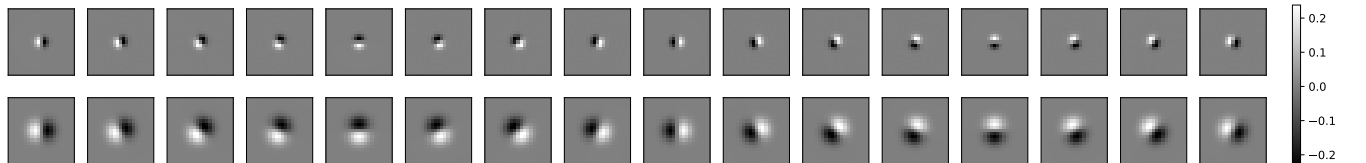


Fig. 1. Filter bank of oriented derivative of gaussian

[†] Luis F. Recalde is PhD student in Robotics Engineering at Worcester Polytechnic Institute lrecalde@wpi.edu

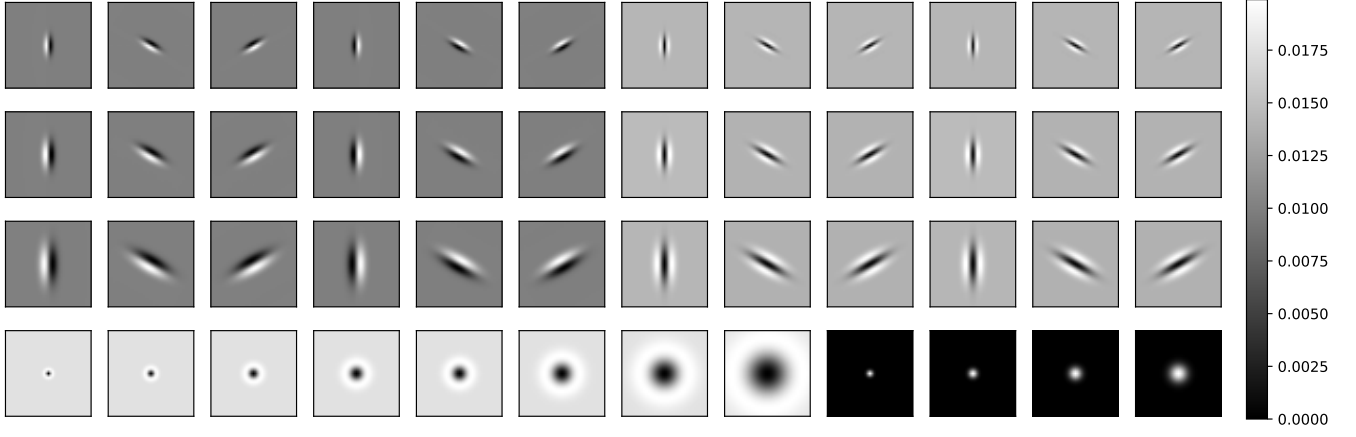


Fig. 2. Filter bank of a small Leung-Malik

and the LOG can be formulated as:

$$H_{\nabla}(x, y, \sigma_x, \sigma_y) = \left(\frac{x^2}{\sigma_x^4} + \frac{y^2}{\sigma_y^4} - \frac{1}{\sigma_x^2} - \frac{1}{\sigma_y^2} \right) H(x, y, \sigma) \quad (5)$$

where $H_{\nabla} = H_{xx} + H_{yy}$. Therefore, the set of the LM filter can be defined by the following:

$$\mathcal{F}_{LM} = \mathcal{F}_x \cup \mathcal{F}_{xx} \cup \mathcal{F}_{\nabla} \cup \mathcal{F}_H \quad (6)$$

where $\mathcal{F}_{xx}$ and $\mathcal{F}_{\nabla}$ denote the sets corresponding to the second-order derivative and the Laplacian of the Gaussian function, respectively. Finally, $\mathcal{F}_H$ constitutes a set of Gaussian functions.

A small LM filter bank, with the values of $\sigma \in \{1, \sqrt{2}, 2, 2\sqrt{2}\}$, is depicted in Fig. 2. It is important to note that this work considers only the first three scales of σ for the first and second order derivatives, where $\sigma_x = \sigma$ and $\sigma_y = 3\sigma_x$. Furthermore, the Gaussian employs all four scales of σ , while the LOG is applied with the following configuration ($\sigma_x = \sigma, \sigma_y = \sigma$) and ($\sigma_x = 3\sigma, \sigma_y = 3\sigma$).

A large LM filter employing $\sigma \in \{\sqrt{2}, 2, 2\sqrt{2}, 4\}$ is presented in Fig. 3. This set of filters follows the same

procedure for the small LM; but only with a different set of scales.

3) *Gabor Filters*: The design of Gabor filters is inspired by the filtering mechanisms in the human visual system. A Gabor filter consists of a Gaussian kernel that is modulated by a sinusoidal function, this formulation is presented as follow:

$$G(x, y, \sigma_x, \sigma_y, f) = H(x, y, \sigma_x, \sigma_y) \cos(2\pi f x) \quad (7)$$

where f defines the frequency. The Gabor filter bank considering $\sigma \in \{5, 6, 7, 8\}$ and $N = 16$ is presented in Fig. 4

B. Creation of Texture, Brightness, and Color Maps

1) *Texture Map*: Based on the previously presented filter banks, these filters can be applied to images to generate $\mathbf{\Pi}$ responses that encode texture properties. This representation can be simplified by replacing the $\mathbf{\Pi}$ tensor with a discrete ID matrix. This simplification is achieved by clustering the $\mathbf{\Pi}$ responses into K clusters using the KMeans clustering algorithm. The texture map can be obtained by setting the

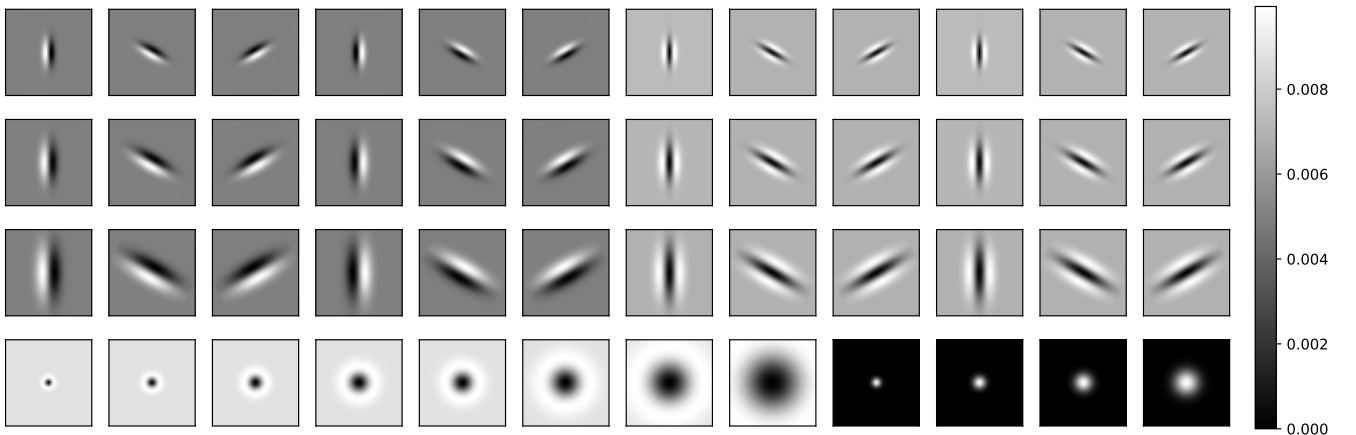


Fig. 3. Filter bank of a large Leung-Malik

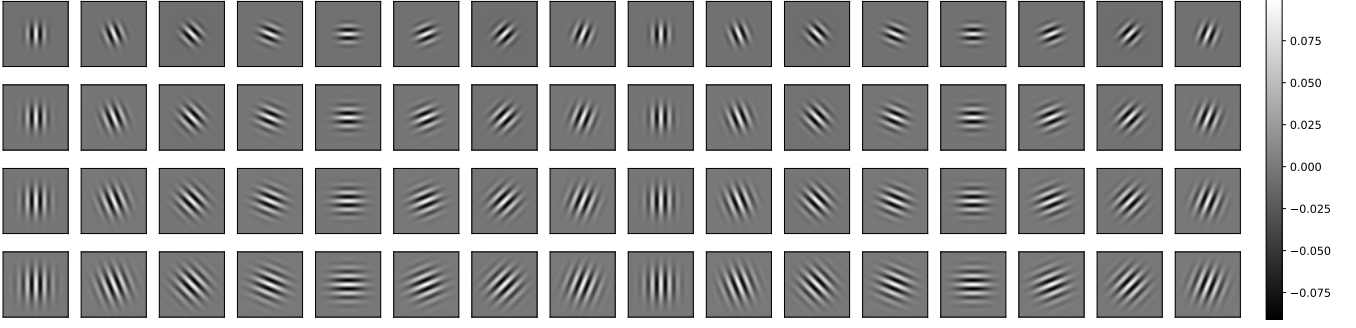


Fig. 4. Gabor filters

number of cluster as $K = 64$. The cluster output matrix can be defined as $\mathcal{T}$, and it is shown in Fig. 5.

2) *Brightness Map*: The brightness map is generated using KMeans clustering on the grayscale image; this work considers an appropriate number of clusters to be $K = 16$. The results of the clustering can be written as $\mathcal{B}$ and are depicted in Fig. 5.

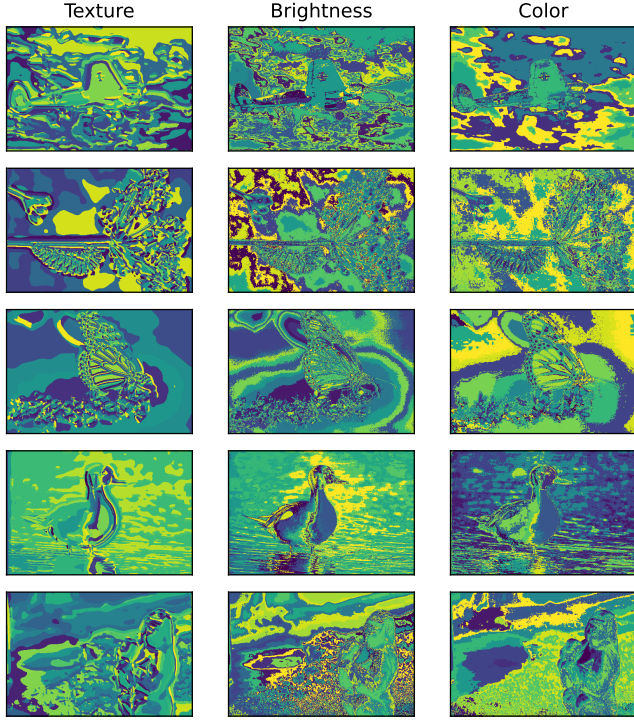


Fig. 5. Texture Map $\mathcal{T}$, brightness map $\mathcal{B}$ and color map $\mathcal{C}$ for 5 images in BSDS500 [4]

3) *Color Map*: The concept of a color map is to effectively represent variations in color within an image. To achieve this, KMeans clustering can be used to group color values, considering the three values per pixel associated with hue, saturation, and value (HSV). This color space is robust to variations in luminance, making it a popular choice for color detection. The results of the clustering can be written as $\mathcal{C}$ and are shown in Fig. 5.

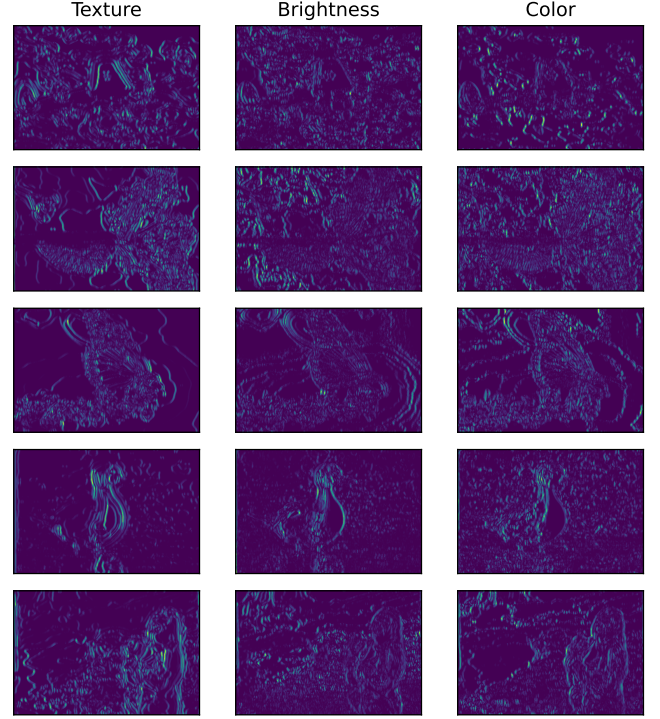


Fig. 6. Texture Map gradient $\mathcal{T}_g$, brightness map gradient $\mathcal{B}_g$ and color map gradient $\mathcal{C}_g$ for 5 images in the BSDS500 dataset [4]

C. Texture, Brightness, and Color Gradients

To compute the gradients of the maps presented in Fig. 5, it is necessary to calculate the chi-square distance χ^2 . This work approximates χ^2 using pairs of half-disk masks, as shown in Fig. 7.

Since the half-disk filters generate different scales and orientations in Fig. 7, they provide a set of localized gradient measurements that capture the rate of change in texture or brightness distributions across multiple scales and angles. Consequently, the chi-square distance can be utilized to compare the texture, brightness, and color distributions. The chi-square distance between the filtered images derived from the half-disk filters can be formulated as follows:

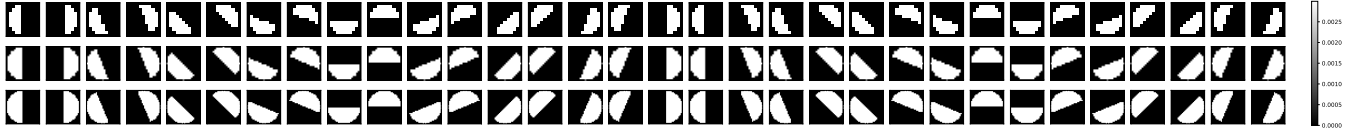


Fig. 7. Half-disk mask filters

$$\chi^2(g, h) = \frac{1}{2} \sum_{i=1}^K \frac{(g_i - h_i)^2}{g_i + h_i} \quad (8)$$

where g_i represents the filtered image derived from one of the paired half-disk masks, and f_i denotes the corresponding filtered image from the remaining pair. The parameter K specifies the number of clusters defined for each map. The gradient for each map is defined by $\mathcal{T}_g$, $\mathcal{B}_g$, and $\mathcal{C}_g$, as illustrated in Fig. 6.

D. Pb-lite output

This section presents the result of the Pb-lite algorithm, this formulation combines the information from the baseline method Sobel and Canny resulting in the following:

$$Pb_{edges} = \left(\frac{\mathcal{T}_g + \mathcal{B}_g + \mathcal{C}_g}{3} \right) \odot (\alpha_1 \text{canny} + \alpha_2 \text{sobel}) \quad (9)$$

where $\odot$ denotes the Hadamard product, while $(\alpha_1, \alpha_2) \in \mathbb{R}$ are scalars that must sum 1.

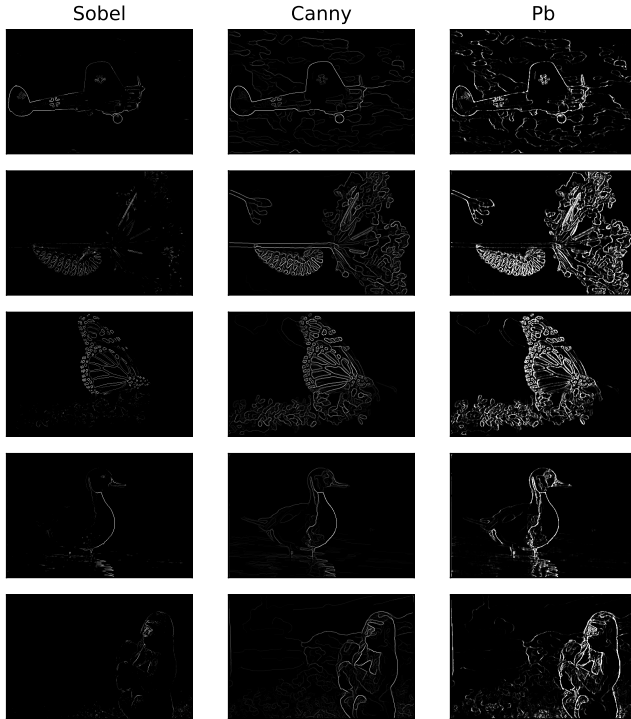


Fig. 8. A comparison between the results obtained by Sobel and Canny and those provided by Pb-lite algorithm

E. Analysis

The Pb-lite algorithm achieves robust boundary detection by integrating local and global image information, such as texture, brightness, and color. This approach improves its performance, particularly in complex scenarios with varying luminance. Furthermore, the algorithm incorporates a bank of filters that generate multiple scales and orientations, significantly increasing its effectiveness. The results obtained using the Pb algorithm appear promising for application in image segmentation procedures, addressing another critical challenge in computer vision. Finally, it should be noted that in order to preserve the intensity of the image and the probability density of the filter banks, normalization is required. The results obtained from Sobel, Canny, and Pb-lite are shown in Fig. 8.

II. PHASE 2: DEEP DIVE ON DEEP LEARNING

Numerous studies in deep learning and computer vision have highlighted its advantages over traditional geometric formulations in tasks such as image classification, segmentation, and odometry. This section explores the implementation of different neural network architectures and compares them based on criteria such as the number of parameters, training and testing accuracies, and other performance metrics. A short analysis is provided to explain why certain architectures outperform others, offering insights into their strengths and limitations.

The neural network architectures were trained and tested on the well-known CIFAR-10 dataset, consisting of 50,000 training images and 10,000 testing images. The training and testing processes were conducted on a computer equipped with 32 GB of RAM and an NVIDIA GTX 4070 graphics card with 8 GB of memory, running the Ubuntu 22.04 operating system.

A. First Neural Network

In this section, the first neural network of this work is presented. This architecture consists of two convolution layers, three fully connected layers, and ReLU activation functions. A detailed representation of this structure is shown in Fig. 9. The parameters used in the training process for this neural network are outlined in Table I, with a training duration of 5[*min*]. The values of the cost function and the precision during the training and validation phases are shown in Fig. 10.

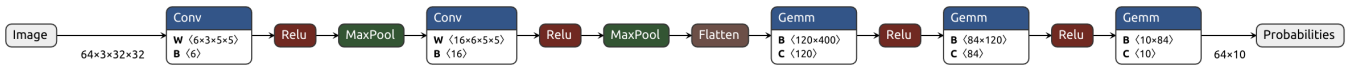


Fig. 9. First neural network architecture

TABLE I
FIRST NEURAL NETWORK PARAMETERS

Learning rate	Batch Size	Epochs	Optimizer
$\eta = 10^{-3}$	$m = 64$	$N = 30$	<i>ADAM</i>
Lost function	Number parameters	Decay	
cross-entropy	62K	$\lambda = 0.0$	

During testing and training, this structure is capable of achieving the precision of 63.38% and 88.27%, respectively. The confusion matrices of the trained model on the training

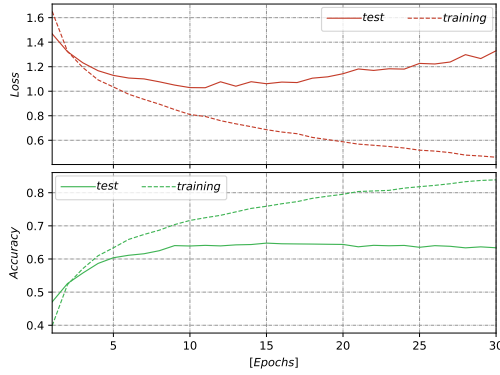


Fig. 10. Values of the cost function during the training and testing phases of the first neural network

and testing data are depicted in Fig. 11 and 12, facilitating a comprehensive understanding of neural network performance in image classification and offering detailed insights into the model's handling of each class.

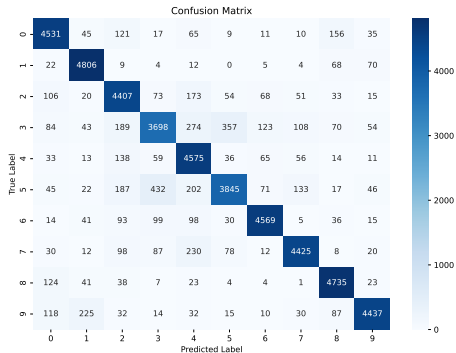


Fig. 11. Training confusion matrix for the first neural network

B. Improving Accuracy of the neural network

This section presents the formulation of improvements for the previously introduced neural network. One of the primary steps to significantly enhance the training process is

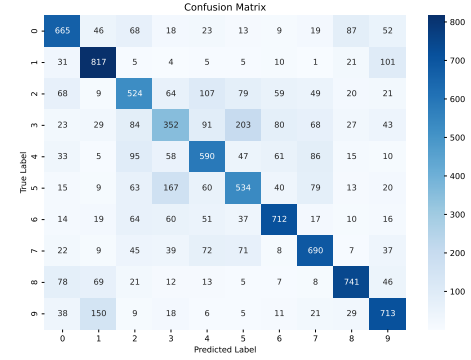


Fig. 12. Testing confusion matrix for the first neural network

TABLE II
IMPROVED NEURAL NETWORK PARAMETERS

Learning rate	Batch Size	Epochs	Optimizer
$\eta = 5 \times 10^{-3}$	$m = 64$	$N = 30$	<i>SGD</i>
Lost function	Number parameters	Decay	
cross-entropy	1M	$\lambda = 0.05$	

normalizing the images. Instead of having an image where its pixel values lie in the range $\mathbf{A} \in [0, 255]$, normalization adjusts these values to $\mathbf{A} \in [-1, 1]$. Furthermore, it is also possible to increase the data set considering operations on images such as random horizontal flips and rotations; these operations are explained in detail in [5].

The parameters used in the neural network training process are detailed in Table II, considering that the training time for this neural network was approximately 10[*min*]. In addition, batch normalization was incorporated into the network to enhance training speed, stability, and overall performance. The new structure of the neural network is presented in Fig.

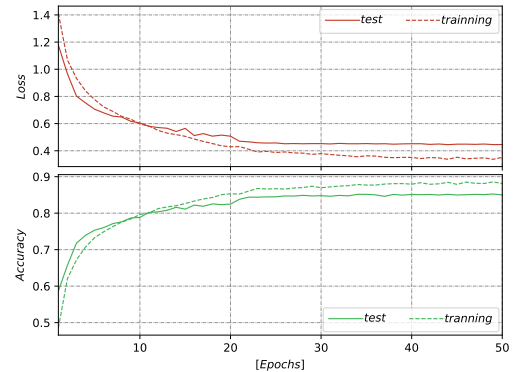


Fig. 13. Values of the cost function during the training and testing phases of the improved neural network

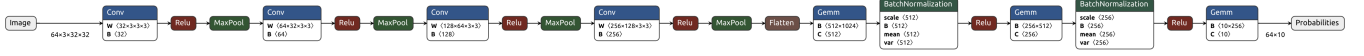


Fig. 14. Improved neural network architecture

14.

The metrics for the cost function and accuracy throughout the training and testing phases are presented in Fig. 13. During both the testing and training phases, this configuration is able to reach precisions of 74.01% and 77.66%, respectively.

Furthermore, the confusion matrix obtained from the training process is depicted in Fig. 15, where there is a notorious improvement of this structure.

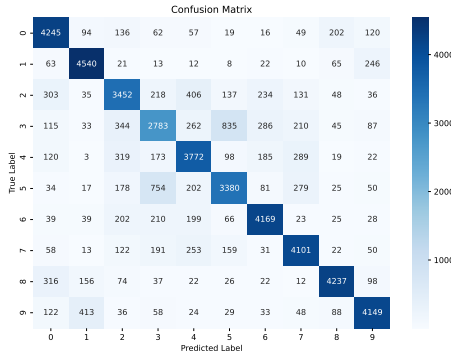


Fig. 15. Training confusion matrix for the improved neural network

Finally, the confusion matrix obtained from the trained neural network through the testing data is shown in Fig. 16.

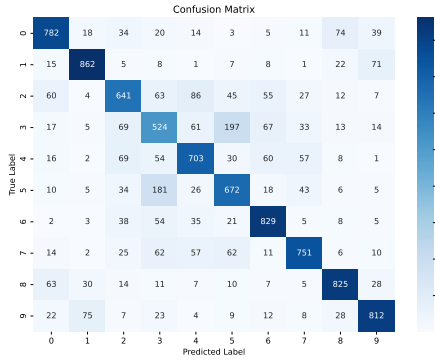


Fig. 16. Testing confusion matrix for the improved neural network

C. ResNet, ResNeXt and DenseNet Implementations

1) *ResNet*: In order to increase the accuracy of the image classification process, this section presents the implementation of the neural network structures of ResNet, ResNeXt, and DenseNet. The ResNet architecture implemented in this work is shown in Fig. 17.

The parameters assigned to this neural network are shown in Table III. The time required to train this architecture was approximately 40[*min*]. Recall that this work implements a small version of the original Resnet.

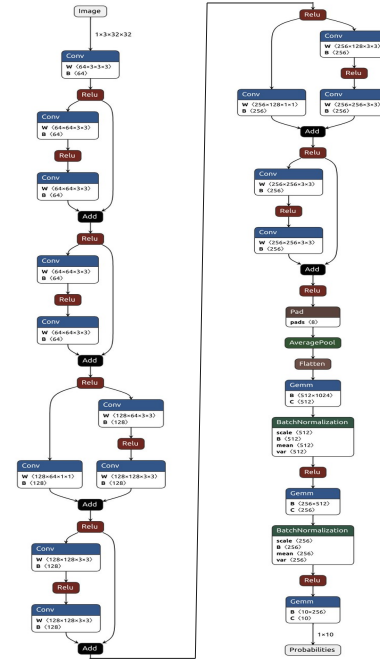


Fig. 17. Resnet architecture

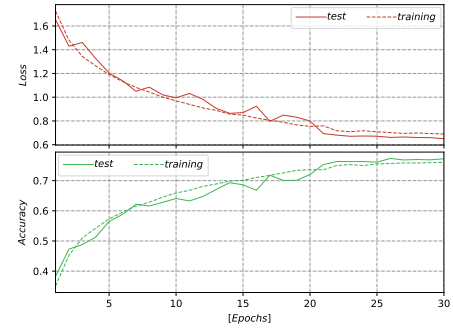


Fig. 18. Values of the cost function during the training and testing phases of the ResNet Neural Network

The values of the cost function and accuracy of the ResNet architecture for training and testing are presented in Fig. 18. These results demonstrate the advantage of this architecture, where shortcuts between layers allow gradients to flow through the network directly without being scaled by intermediate layers.

This configuration results in accuracies of 77.25% for the testing phase and 79.16% for the training phase. The confusion matrices for both training and testing are illustrated in Fig. 19 and Fig. 20.

2) *ResNeXt*: On the other hand, the ResNeXt architecture implemented in this work is shown in Fig. 21. This formulation groups convolutions, significantly decreasing compu-

TABLE III
RESNET PARAMETERS

Learning rate	Batch Size	Epochs	Optimizer
$\eta = 5 \times 10^{-3}$	$m = 64$	$N = 30$	SGD
Loss function	Number parameters	Decay	
cross-entropy	$1.8M$	$\lambda = 0.05$	

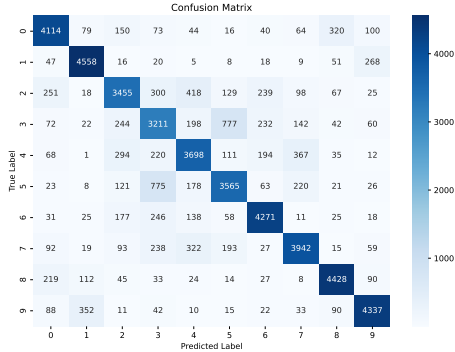


Fig. 19. Training confusion matrix for the ResNet neural network

tational complexity. The parameters used during the training process can be found in Table IV; where the time required to train this formulation was approximately $3[Hours]$.

TABLE IV
RESNEXT PARAMETERS

Learning rate	Batch Size	Epochs	Optimizer
$\eta = 5 \times 10^{-3}$	$m = 64$	$N = 50$	SGD
Loss function	Number parameters	Decay	
cross-entropy	$2.1M$	$\lambda = 0.05$	

The cost function values and the network accuracy are shown in Fig. 22. These results closely resemble those presented in Fig. 18, as ResNeXt builds on the ResNet architecture by incorporating parallel convolutional paths within each block. While ResNet improves training stability and performance through residual connections, ResNeXt further improves these aspects by employing grouped convolutions that split computations into multiple parallel branches.

This structure is able to achieve an accuracy of 78.77% during testing and a value of 80.93% within training. Finally, the confusion matrices corresponding to the training and testing phases are depicted in Fig. 23 and Fig. 24, respectively.

3) *DenseNet*: The DenseNet architecture is founded on the principle of interconnecting each layer with every other layer within the network, thus creating direct connections between layers in a feed-forward manner. This study employs the neural network structure implemented in [6], due to the intricate nature of the formulation of this architecture. The structure of this formulation is depicted in Fig. 25.

The parameters used for the training process are shown in Table V, mentioning that the require time to train this architecture was approximately 5[*hours*].

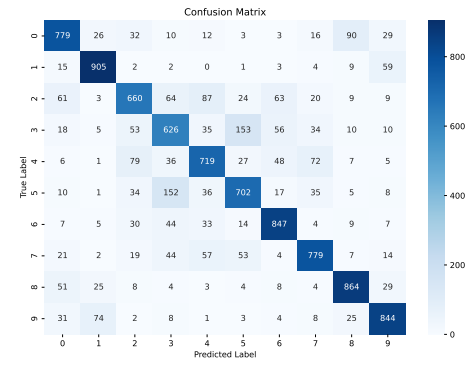


Fig. 20. Testing confusion matrix for the ResNet neural network

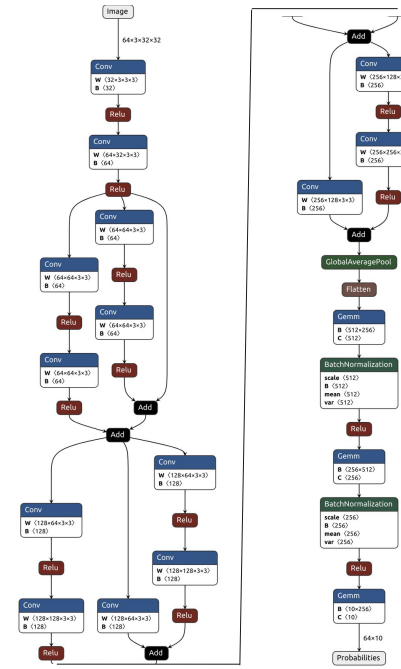


Fig. 21. ResNext architecture

Furthermore, Fig. 26 illustrates the outcomes of the cost function and accuracy metrics for both training and testing phases. Due to the design of this neural network, it achieves accuracy rates of 83.07% on testing and 87.56% on training experiments.

Finally, the confusion matrices associated to this structure are shown in Fig. 27 for training experiments and Fig. 28 for testing experiments respectively.

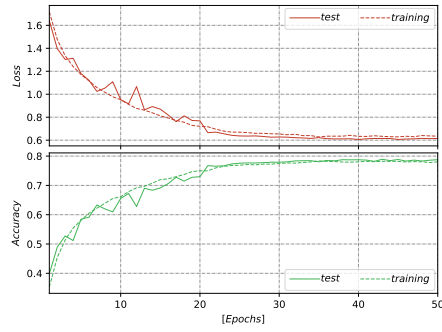


Fig. 22. Values of the cost function during the training and testing phases of the ResNext Neural Network

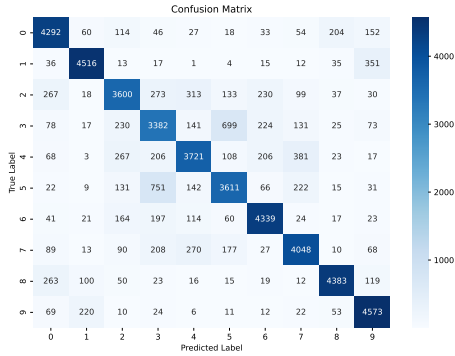


Fig. 23. Training confusion matrix for the ResNext neural network

D. Analysis

This section provides a brief analysis of the different neural network structures implemented in the previous sections. The initial neural network was expected to have the lowest accuracy compared to the other architectures. However, by incorporating data augmentation, batch normalization, and increasing the number of layers, the improved neural network achieved better accuracy performance.

Despite these improvements, a significant disadvantage of this approach is the potential for gradients to vanish through deep layers. To overcome this issue, the formulations presented in ResNet and ResNeXt introduce shortcuts between layers and utilize parallel structures, effectively addressing the vanishing gradient problem.

Finally, the Dense Neural Network demonstrated the best performance in terms of accuracy among all evaluated architectures. This superior performance is attributed to its structure, which establishes direct connections between layers in a feed-forward manner, enhancing information flow throughout the network. However, although this architecture exhibits the highest accuracy compared to the others, it also requires significantly more time for inference. This represents a drawback of this structure, particularly if there is an increase in image size. More details about the performance of each architecture can be found in Table. VI

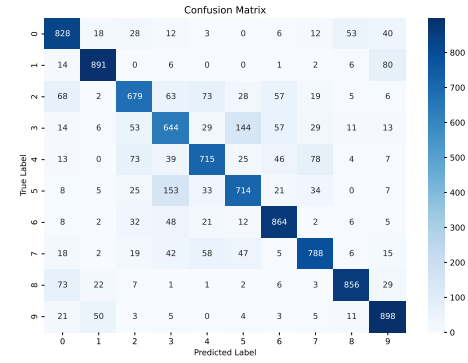


Fig. 24. Testing confusion matrix for the ResNext neural network

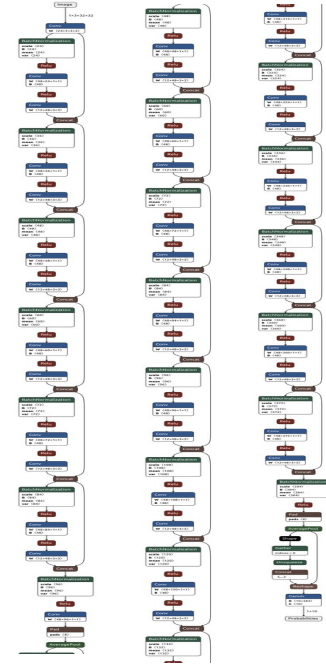


Fig. 25. DenseNet architecture

REFERENCES

- [1] M. Thompson, *Leonardo*, vol. 7, no. 4, pp. 370–370, 1974. [Online]. Available: <http://www.jstor.org/stable/1573081>
- [2] J. Canny, “A computational approach to edge detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986.
- [3] P. Arbeláez, M. Maire, C. Fowlkes, and J. Malik, “Contour detection and hierarchical image segmentation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 5, pp. 898–916, 2011.
- [4] D. Martin, C. Fowlkes, D. Tal, and J. Malik, “A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics,” in *Proc. 8th Int’l Conf. Computer Vision*, vol. 2, July 2001, pp. 416–423.
- [5] P. Team. (2025) Torchvision transforms documentation. [Online]. Available: <https://pytorch.org/vision/main/transforms.html>
- [6] K. Liu, “Pytorch cifar models,” <https://github.com/kuangliu/pytorch-cifar/tree/master>, 2017.

TABLE V
DENSENET NEURAL NETWORK PARAMETERS

Learning rate	Batch Size	Epochs	Optimizer
$\eta = 5 \times 10^{-3}$	$m = 64$	$N = 50$	SGD
Lost function	Number parameters	Decay	
cross-entropy	$1.1M$	$\lambda = 0.05$	

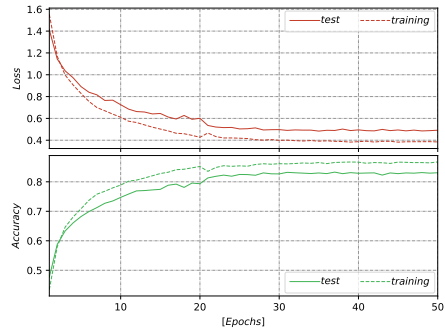


Fig. 26. Values of the cost function during the training and testing phases of the DenseNet Neural Network

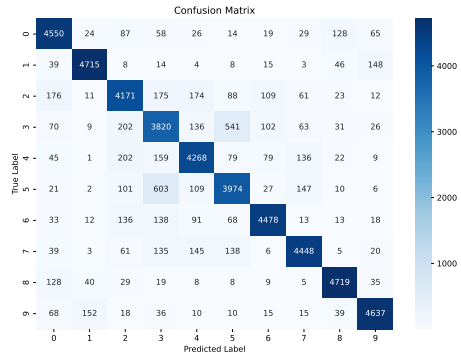


Fig. 27. Training confusion matrix for the DenseNet neural network

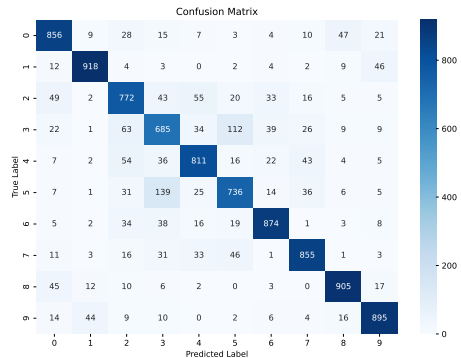


Fig. 28. Testing confusion matrix for the DenseNet neural network

TABLE VI
PERFORMANCE METRICS: COMPARISON BETWEEN THE IMPLEMENTED NEURAL NETWORKS

	Accuracy		Optimization variables	Inference Time
	Training	Testing	Testing	Testing
First Neural Network	88.06%	63.38%	62 <i>K</i>	0.9[<i>ms</i>]
Improved Neural Network	77.66%	74.01%	1 <i>M</i>	0.7[<i>ms</i>]
ResNet	79.16%	77.25%	1.8 <i>M</i>	3.8[<i>ms</i>]
ResNext	80.93%	78.77%	2.1 <i>M</i>	6.51[<i>ms</i>]
DenseNet	87.56%	83.07%	1 <i>M</i>	6.9[<i>ms</i>]